Session types R context free

Many authors among which we find Bernardo Almeida, Diana Costa, Andreia Mordido, Diogo Poças, Gil Silva, Peter Thiemann, and Vasco T. Vasconcelos,
University of Lisbon, University of Freiburg

Dagstuhl seminar on "Behavioral Types for Resilience", 8-13 Feb, 2026

back to the classics

Types for Dyadic Interaction

Kohei Honda

kohei@mt.cs.keio.ac.jp

Department of Computer Science, Keio University

3-14-1 Hiyoshi, Kohoku-ku, Yokohama, 223, Japan

Abstract

We formulate a typed formalism for concurrency where types denote freely composable structure of dyadic interaction in the symmetric scheme. The resulting calculus is a typed reconstruction of name passing process calculi. Systems with both the explicit and implicit typing disciplines, where types form a simple hierarchy of types, are presented, which are proved to be in accordance with each other. A typed variant of bisimilarity is formulated and it is shown that typed β-equality has a clean embedding in the bisimilarity. Name reference structure induced by the simple hierarchy of types is studied, which fully characterises the typable terms in the set of untyped terms. It turns out that the name reference structure results in the deadlock-free property for a subset of terms with a certain regular structure, showing behavioural significance of the simple type discipline.

Concur 1993

Inflexion...

An Interaction-based Language and its Typing System

Kaku Takeuchi Kohei Honda Makoto Kubo

Department of Computer Science
Keio University

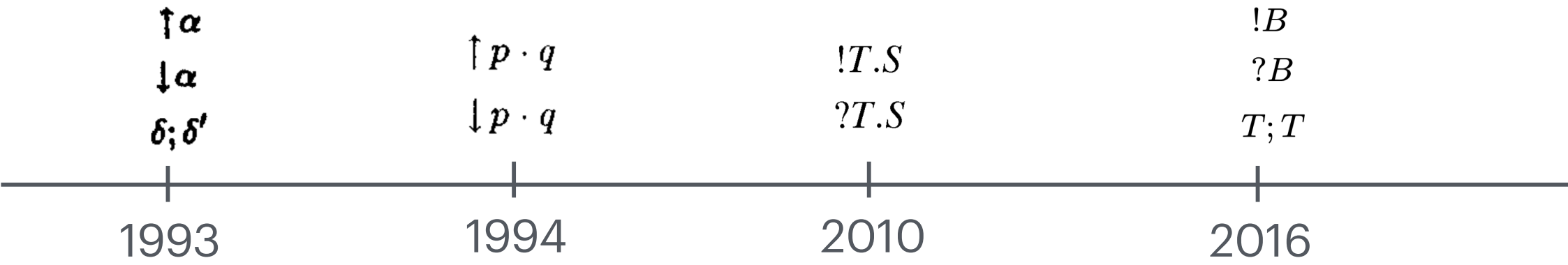
3-14-1, Hiyoshi, Kohoku-ku, Yokohama 223, Japan

{kaku,kohei,kubo}@mt.cs.keio.ac.jp

Parle 1994

Abstract. We present a small language $\mathcal{L}$ and its typing system, starting from the idea of interaction, one of the important notions in parallel and distributed computing. $\mathcal{L}$ is based on, apart from such constructs as parallel composition and process creation, three pairs of communication primitives which use the notion of a *session*, a semantically atomic chain of communication actions which can interleave with other such chains freely, for high-level abstraction of interaction-based computing. The three primitives enable programmers to elegantly describe complex interactions among processes with a rigorous type discipline similar to ML [4]. The language is given formal operational semantics and a type inference system, regarding which we prove that if a program is well-typed in the typing

Session types over time



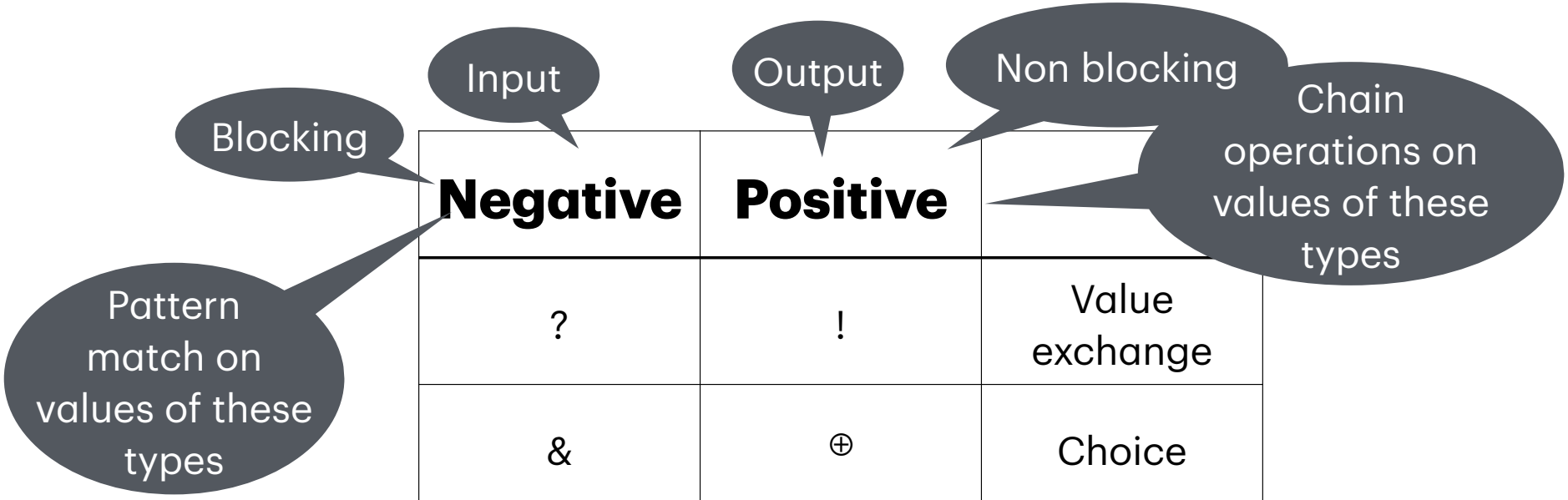
Types for dyadic interaction

An Interaction-based Language and its Typing System

Linear Type Theory for Asynchronous Session Types

Context-free Session Types

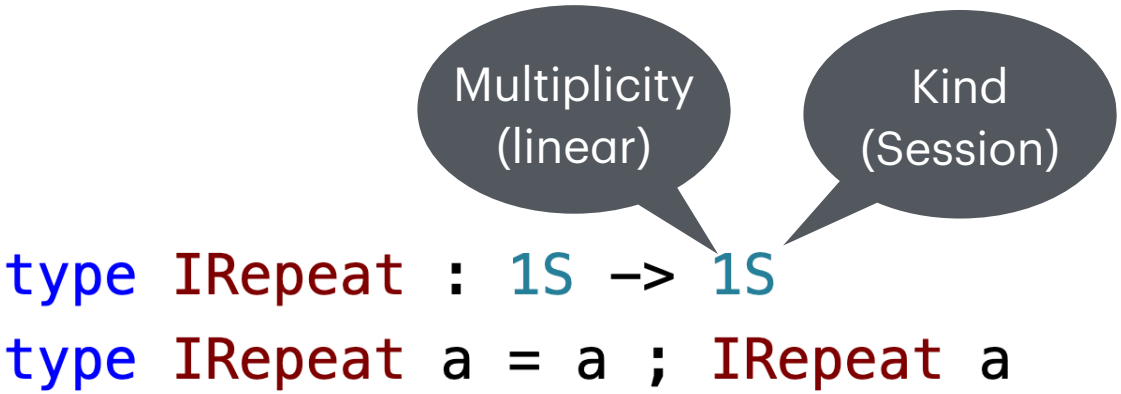
the elements of dyadic interaction



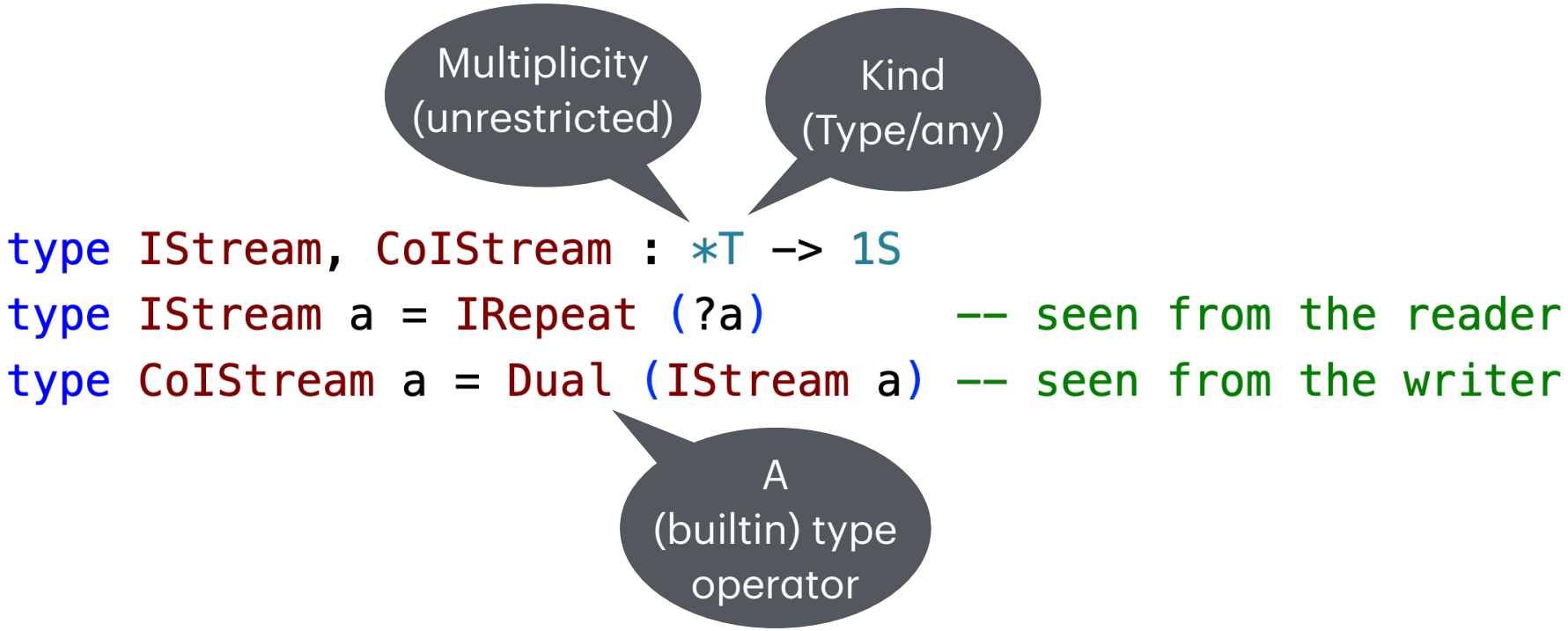
Negative	**Positive**	
?	!	Value exchange
&	⊕	Choice
∀	∃	Type exchange
Wait	Close	Channel closing

Programming with CFSTs

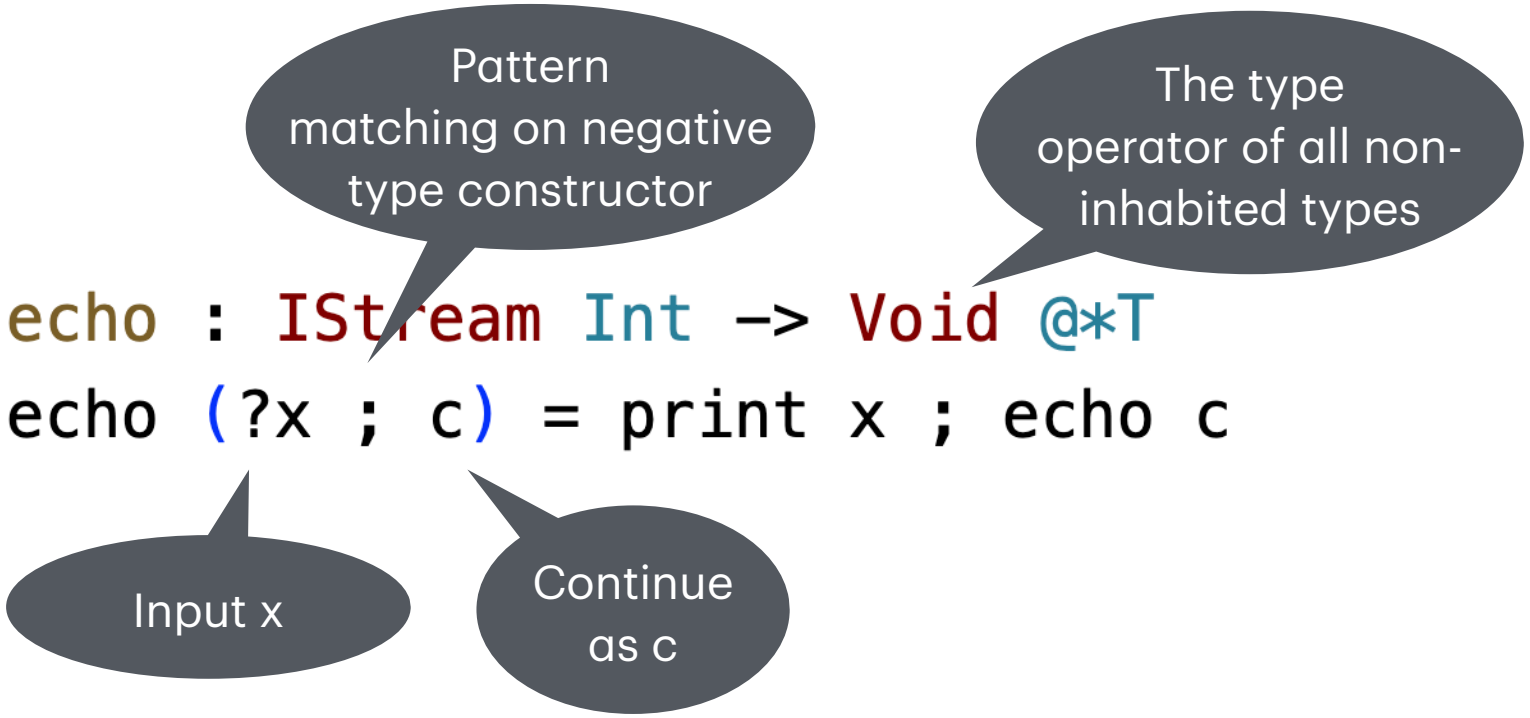
Infinitely repeating some action a



Infinite streams of some type a



A consumer of type IStream Int



A consumer of type CoIStream

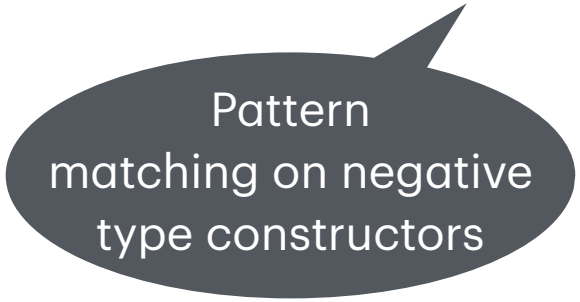

```
ints : Int -> CoIStream Int -> Void @*T
ints n c =
  c |> send n |> ints (n + 1)          -- preferred
  ints (n + 1) (send n c)              -- the functional way
  let c = send n c in ints (n + 1) c -- alternative
```

Finite or infinite streams

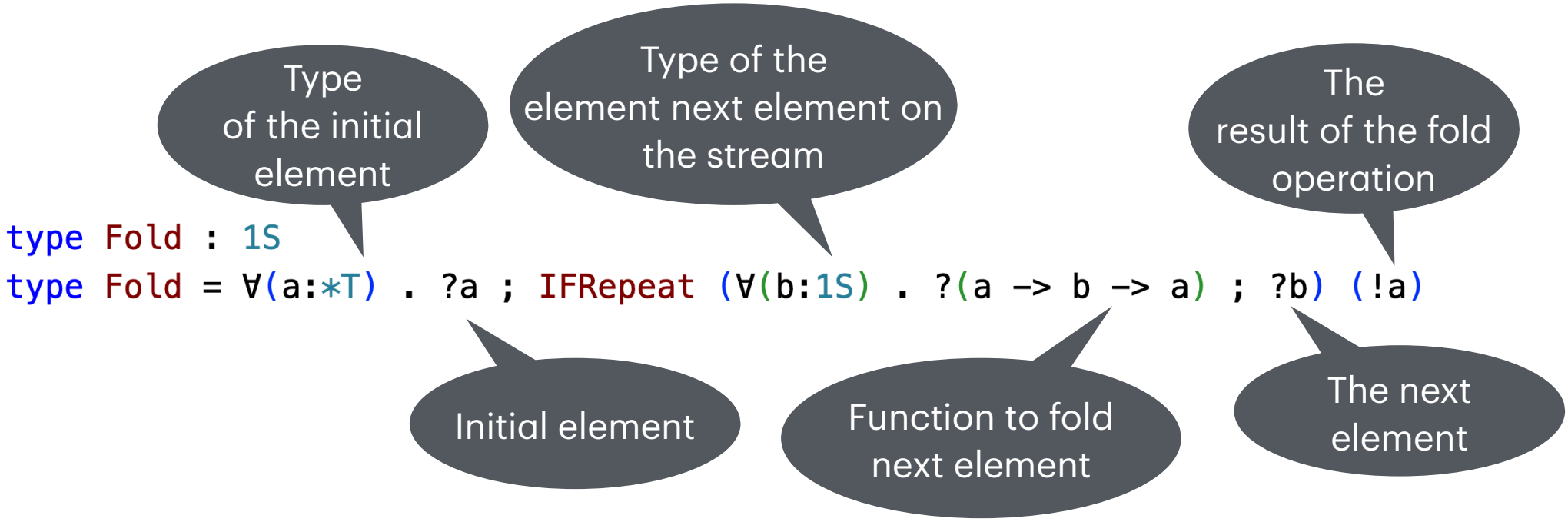
```
type IFRepeat : 1S -> 1S -> 1S
type IFRepeat a b = IRepeat (&{More: a, Done: b ; Wait})        -- unfold
--  ≃ &{More: a, Done: b ; Wait}; IFRepeat a b                  -- distributivity
--  ≃ &{More: a ; IFRepeat a b, Done: b ; Wait ; IFRepeat a b} -- Wait is absorbing
--  ≃ &{More: a ; IFRepeat a b, Done: b ; Wait}                 -- fold
--  ≃ μc.&{More: a ; c, Done: b ; Wait}
```

A consumer of type IFRepeat (?a) Skip

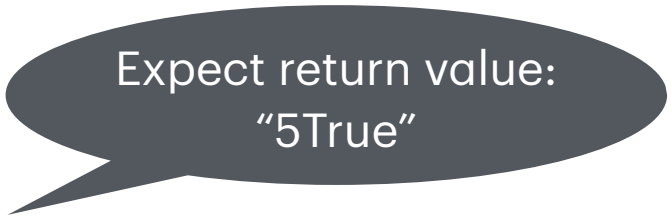
```
length : IFRepeat (?a) Skip -> Int
length (&Done Wait) = 0
length (&More (?_ ; c)) = 1 + length c
```



Folding a stream of heterogeneous values



A consumer for type Dual Fold




```
showStream : Dual Fold -> String
showStream c =
  let (x, c) = c |> sendType @String |> send ""
                 |> select More |> sendType @Int
                 |> send (\(x:String) (y:Int) -> x ++ showInt y) |> send 5
                 |> select More |> sendType @Bool
                 |> send (\(x:String) (y:Bool) -> x ++ showBool y) |> send True
                 |> select Done
                 |> receive
  in close c ; x
```

A consumer of type Fold

```
fold : Fold -> ()
fold (@a, (?x ; c)) = fold' @a x c
  where
    fold' x (&Done c) = c |> send x |> wait
    fold' x (&More (@b, (?f ; ?y; c))) = fold' (f x y) c
```

FreeST 3.2

https://freest-lang.github.io/

Up and running

FreeST 5.0

https://github.com/freest-lang/freest

Coming soon